

API SPECIFICATION DOCUMENT

PART-2: AUTHENTICATION, AUTHORIZATION & API SECURITY

School ERP

Spring Security | JWT | Redis

Prepared by: API Architecture Team

Document Version: 1.0

Date: 29 July 2026

Classification: Confidential — Engineering Team Only

Preamble

This document is Part-2 of the API Specification Document (ASD) v1.0 for the School ERP System, continuing directly from Part-1: API Foundation. It defines authentication, authorization, session, and security standards specifically at the API-contract level — how a client of the API observes and interacts with the mechanisms already fixed in Master Development Blueprint (MDB) Part-2 Sections 11-13 and 19-20.

This document does not re-derive the underlying security mechanism — Spring Security, JWT signing, Redis-backed refresh-token state, and the Role Permission model all remain exactly as already fixed in MDB Part-2 and Appendix-E Field 38. This ASD Part narrows that mechanism to its API-consumer-facing behavior: what a client must send, what a client receives, and what a client can expect from every API Group already catalogued in Appendix-K Section 18.

This document contains no source code, no controller code, no security configuration, no JSON, no API endpoints, and no sequence diagrams — every standard is stated as a rule only.

11. Authentication Standards

Fixes the API-contract view of the Authentication Architecture already established in MDB Part-2 Section 11 and Appendix-K Section 4.

- Every API operation, except the login operation itself and the Admission public-inquiry intake already scoped in Appendix-L Section 19.1, requires a valid JWT access token (Section 13) — there is no API-level exception granted ad hoc per module or per screen.
- A failed authentication attempt returns the Authentication Failure response category already fixed in ASD Part-1 Section 10 / Appendix-K Section 8, without revealing whether the supplied identifier (username/email) exists, consistent with MDB Part-2 Section 11's account-enumeration protection.
- Account lockout state (five consecutive failures, NFR-PWD-003) is reflected to the API consumer as a distinct Authentication Failure sub-state, distinguishable from an incorrect-credential failure, so a legitimate client can present an appropriate "account locked" message rather than repeatedly retrying.
- Multi-Factor Authentication remains architecturally supported but not active at go-live (NFR-AUTHN-002, MDB Part-2 Section 11) — the API contract reserves an extension point for an additional authentication step without requiring a breaking version change (ASD Part-1 Section 7) when MFA is eventually enabled.

12. Authorization Standards

Fixes the API-contract view of the RBAC model already established in MDB Part-2 Section 12 and Appendix-E Field 38.

- Every API Group's Create/Read/Update/Delete/Approve/Reject/Export/Print operations are individually gated by the Role Permission matrix already fixed in Appendix-E Field 38 and Appendix-I Section 8.3 — an API consumer authorized for Read on an API Group is not implicitly authorized for Update on the same Group.
- An authorization failure returns the Authorization Failure response category (ASD Part-1 Section 10) without revealing whether the target resource exists, consistent with MDB Part-2 Section 12's information-disclosure protection — this applies uniformly whether the failure is role-based or record-scoped (e.g., a Parent requesting a non-linked student's record).
- Record-level and field-level restrictions that cannot be expressed as a simple role check (a Teacher's own Section, a Parent's own linked children, Sensitive/PII field visibility per Appendix-G/H's Sensitive Data Matrix) are enforced identically regardless of which API Group or version is used — these restrictions are a property of the data and the caller, not of the specific endpoint.
- The Finance-Team-exclusive refund authority (BR-FEE-002) and other single-role-exclusive Business Rules remain enforced exactly as already fixed in MDB Part-2 Section 12, with no API-layer path that bypasses that exclusivity.

13. JWT Access Token Standards

Fixes the API-contract view of the JWT already defined in MDB Part-2 Section 13.

Aspect	API-Contract Standard
Transport	Presented via the standard Authorization header on every request requiring authentication (Section 11)
Claims Content	Identity, role, and authority set (Section 12) only — no Sensitive/PII payload data beyond identity is ever embedded, consistent with MDB Part-2 Section 13
Expiry Visibility	A client can determine an access token's remaining validity without a dedicated introspection call, supporting proactive refresh (Section 14) before expiry
Expired-Token Handling	A request with an expired access token returns the Authentication Failure category (Section 11), directing the client to the Refresh Flow (Section 14) rather than to re-login, unless the refresh token has also expired or been revoked
Signature Validation	Every request's token signature is validated before any Controller logic executes, per MDB Part-2 Section 11 — this remains unchanged at the API-contract level

14. Refresh Token Standards

Fixes the API-contract view of the Refresh Token lifecycle already defined in MDB Part-2 Section 13, using Redis-backed server-side tracking.

- A dedicated refresh operation exchanges a valid, non-revoked refresh token for a new access token without requiring the credentials used in Section 11 — this operation is the only API interaction permitted without a currently-valid access token, aside from login itself.
- A revoked or expired refresh token returns the Authentication Failure category and requires the client to return to the full login flow (Section 11) — there is no silent, automatic re-authentication path bypassing credential entry.
- Refresh token revocation (logout, password change, access revocation per BR-HR-002) takes effect immediately at the API layer — a refresh attempt using a token revoked moments earlier is rejected, not honored due to caching or propagation delay, consistent with the Redis-backed tracking already fixed in MDB Part-2 Section 13.
- A client is expected to attempt exactly one refresh per expired-access-token event before falling back to the full login flow — the API does not support an indefinite refresh-retry loop, bounding the exposure window of a compromised refresh token.

15. Session Management Standards

Fixes the API-contract view of session behavior, extending MDB Part-2 Section 13 and Appendix-F NFR-SESS-001 with explicit lifecycle rules.

- A "session," from the API's perspective, is the lifetime of a refresh token (Section 14) — there is no additional, separate server-side session construct beyond the access/refresh token pair and its Redis-tracked state.
- Idle-timeout behavior (NFR-SESS-001, exact duration Client/Product Decision Required) is enforced by refresh-token expiry, not by a separate idle-detection mechanism — a client that stops making requests naturally loses its session once both tokens expire.
- Concurrent sessions (the same identity authenticated from multiple devices/browsers) are permitted by default, each tracked as an independent refresh token in Redis — revoking one session (e.g., via explicit logout) does not revoke another concurrent session unless the revocation is explicitly system-wide (e.g., BR-HR-002 access revocation, which revokes every session for that identity).
- A password change (Section 11) revokes every existing refresh token for that identity, forcing re-authentication on all devices — this is a deliberate, system-wide session invalidation, not a per-device action.

16. API Security Standards

Fixes the API-contract view of the Security Standards already established in MDB Part-2 Section 19, focused on what the API boundary itself enforces.

Control	API-Boundary Enforcement
Transport Encryption	Every API operation is reachable only over an encrypted transport (MDB Part-2 Section 19); a plaintext request is rejected before reaching the Application Layer
Input Sanitization	Every request payload is validated (ASD Part-1 Section 9) and sanitized against injection/XSS before it reaches Service-layer logic, regardless of client-side validation already performed
Output Encoding	Every response value derived from user-supplied content is encoded appropriately before being included in the response body, preventing stored-XSS propagation through the API
File Upload Boundary	Every file-accepting operation enforces type/size validation and malware scanning (MDB Part-2 Section 18) before the file is considered accepted, consistent with NFR-FUS-001
Error Detail Suppression	System-error responses (ASD Part-1 Section 10) never include internal stack traces, query text, or infrastructure identifiers, regardless of caller role, including IT Admin

17. Rate Limiting Standards

Fixes the API-contract view of Rate Limiting already established in MDB Part-2 Section 14 and Appendix-F NFR-APIP-001.

- Rate limits are applied per authenticated identity (Section 11) for interactive API traffic, and separately per external-integration source (Payment Gateway callbacks, GPS device feeds) for machine-originated traffic, since these two traffic classes have fundamentally different volume and burst characteristics.
- A rate-limited request receives the Rate Limit Exceeded response category already fixed in ASD Part-1 Section 10 / Appendix-K Section 8, with the request rejected before reaching Service-layer logic, consistent with the Redis-backed distributed counting already fixed in MDB Part-2 Section 14.
- Specific numeric thresholds (requests per minute per identity, per integration source) remain Client/Product Decision Required, consistent with the same open item already tracked in Appendix-F NFR-APIP-001 — this document fixes the enforcement mechanism and response contract, not the numeric limit.

18. CORS & CSRF Standards

Fixes cross-origin and cross-site request handling for the React + TypeScript frontend's interaction with the Spring Boot 3 API.

- Cross-Origin Resource Sharing (CORS) is restricted to the specific, known origin(s) serving the React frontend per environment (Appendix-N Section 5) — a wildcard/open CORS policy is not used in any environment, including Development.
- CSRF protection (MDB Part-2 Section 19) applies to state-changing requests that could be influenced by ambient browser credentials; since the primary authentication mechanism is a bearer JWT (Section 13) not automatically attached by the browser, the CSRF exposure surface is narrower than a cookie-session-based API, but the CSRF token convention already fixed in MDB Part-2 Section 19 remains in force wherever a cookie-based credential is used (e.g., an optional "remember me" refresh-token cookie, if adopted).
- Preflight (OPTIONS) requests are handled without requiring authentication (Section 11), consistent with standard CORS behavior, but never expose any information beyond what the CORS policy itself requires.

19. API Audit & Logging Standards

Fixes the API-contract view of the Audit and Logging standards already established in MDB Part-2 Section 20 and DDD Part-3 Section 24.

- Every API request is logged at minimum with its request identifier (ASD Part-1 Section 9), authenticated identity (Section 11), target API Group, operation, and outcome status (ASD Part-1 Section 10) — this logging occurs at the API boundary itself, independent of whether the underlying Service method also writes to AuditLog.
- Every state-changing API operation results in an AuditLog write (MDB Part-2 Section 20, DDD Part-3 Section 24) correlated to the same request identifier used in the API-level log, allowing a single request to be traced end-to-end from API entry to database change.
- Authentication and Authorization failures (Sections 11-12) are logged with the same rigor as successful operations, feeding the account-lockout and unusual-access-pattern monitoring already fixed in Appendix-F NFR-MON-001/NFR-PWD-003.
- Sensitive/PII values are never written to API-level request/response logs in plaintext, consistent with MDB Part-2 Section 20's logging redaction rule — this applies to the API-boundary log independently of the same rule already applying to AuditLog and application logs.

20. API Security Review Checklist

Verified for every new or changed API operation before release, extending the Code Review Checklist already fixed in MDB Part-3 Section 30 with API-specific security items.

#	Checklist Item
1	Operation requires authentication (Section 11) unless explicitly and deliberately public
2	Operation's authorization check (Section 12) matches its Appendix-E Field 38 permission exactly, including any record/field-level restriction
3	No Sensitive/PII field is returned to a caller role not entitled to it (Appendix-G/H Sensitive Data Matrix)
4	Request validation (ASD Part-1 Section 9) covers every mandatory field and format rule already fixed in Appendix-G
5	Idempotency (ASD Part-1 Section 4) is implemented for any Payment, Admission-confirmation, or similarly consequence-sensitive operation
6	Rate limiting (Section 17) applies to any newly-introduced high-frequency or externally-triggered operation
7	CORS configuration (Section 18) has not been loosened beyond the approved origin list for any environment
8	API-level and AuditLog logging (Section 19) are both present and correlated by request identifier
9	Error responses (ASD Part-1 Section 10) reveal no internal detail, for every failure branch including unhandled exceptions
10	Response payload uses a DTO (ASD Part-1 Section 10) with no JPA entity or unreviewed field newly exposed